AI Assisted Coding Lab 19.3

Veekshitha Adharasani

2303A52175

Batch – 41

Question 1: Python to C++ Conversion

Prompt:

Convert the given Python class representing a Student into an equivalent C++ class while keeping the same structure and functionality. Make sure to implement constructors properly, use suitable data types for all attributes, and apply correct access specifiers like private and public. Also, include all methods with the same behavior as in the Python version and add a main() function in C++ to demonstrate how the class works. Ensure that the output of the C++ program matches the Python code.

```python
class Student:
    def __init__(self, name, age, marks):
        self.name = name
        self.age = age
        self.marks = marks

    def display(self):
        print("Name:", self.name)
        print("Age:", self.age)
        print("Marks:", self.marks)

# Demonstrate Python class
student_py = Student("Alice", 20, 95)
student_py.display()
```

Code:

```cpp
%%writefile student.cpp
#include <iostream> // Required for input/output operations (e.g., std::cout)
#include <string>   // Required for using std::string

// Define the Student class
class Student {
private:
    // Private members to encapsulate data
    std::string name; // Stores the student's name (string)
    int age;          // Stores the student's age (integer)
    int marks;        // Stores the student's marks (integer)

public:
    // Constructor: Initializes the Student object
    // Takes name, age, and marks as parameters
    Student(std::string name_param, int age_param, int marks_param) {
        name = name_param;   // Assigns the provided name to the object's name
        age = age_param;     // Assigns the provided age to the object's age
        marks = marks_param; // Assigns the provided marks to the object's marks
    }

    // Public method to display student information
    // This method has the same behavior as the Python display method
    void display() {
```

```cpp
    void display() {
        std::cout << "Name: " << name << std::endl;  // Prints the student's name
        std::cout << "Age: " << age << std::endl;    // Prints the student's age
        std::cout << "Marks: " << marks << std::endl; // Prints the student's marks
    }
};

// Main function to demonstrate the Student class
int main() {
    // Create an instance of the Student class using the constructor
    Student student_cpp("Alice", 20, 95);

    // Call the display method to show the student's information
    student_cpp.display();

    return 0; // Indicates successful program execution
}
```

Output:

```
Compilation successful. Running the C++ program:
Name: Alice
Age: 20
Marks: 85
```

Explanation:

We converted the Python Student class into a C++ version because different programming languages are good for different things. Python is fantastic for quickly writing and testing ideas, like our Student class example. However, C++ can be much faster and offers more control over how a program uses computer resources. So, if you have a part of your program, like handling many Student records, that needs to run very quickly or work directly with specific hardware, converting it to C++ helps make it perform better. It's like building a prototype with LEGOs (Python) and then rebuilding the final product with stronger, faster materials (C++) for heavy-duty use

Question 2: Java to Python Function Conversion

Prompt:

Convert the given Java method that checks whether a number is prime into an equivalent Python function. Ensure the logic remains the same while adapting it to Pythonic syntax and style. The function should correctly handle different inputs and return whether a number is prime or not. Also, include sample test cases in Python to verify that the function works correctly for multiple inputs.

```java
public class PrimeCheck {

    public static boolean isPrime(int n) {
        if (n <= 1) {
            return false;
        }
        for (int i = 2; i <= Math.sqrt(n); i++) {
            if (n % i == 0) {
                return false;
            }
        }
        return true;
    }

    public static void main(String[] args) {
        int[] testNumbers = {0, 1, 2, 3, 4, 5, 10, 17, 19, 20};

        for (int num : testNumbers) {
            if (isPrime(num)) {
                System.out.println(num + " -> Prime");
            } else {
                System.out.println(num + " -> Not Prime");
            }
        }
    }
}
```

Code:

```python
import math

def is_prime(num):
    """
    Checks if a number is prime using the same logic as the Java PrimeChecker.
    A number is prime if it's greater than 1 and has no divisors other than 1 and itself.
    """
    if num <= 1:
        return False
    # Iterate from 2 up to the square root of num
    # If num is divisible by any number in this range, it's not prime
    for i in range(2, int(math.isqrt(num)) + 1):
        if num % i == 0:
            return False
    return True

# Sample test cases
print(f"Is 17 prime? {is_prime(17)}")      # Expected: True
print(f"Is 2 prime? {is_prime(2)}")        # Expected: True
print(f"Is 1 prime? {is_prime(1)}")        # Expected: False
print(f"Is 0 prime? {is_prime(0)}")        # Expected: False
print(f"Is -5 prime? {is_prime(-5)}")      # Expected: False
print(f"Is 4 prime? {is_prime(4)}")        # Expected: False
print(f"Is 29 prime? {is_prime(29)}")      # Expected: True
print(f"Is 100 prime? {is_prime(100)}")    # Expected: False
```

Output:

```
Is 17 prime? True
Is 2 prime? True
Is 1 prime? False
Is 0 prime? False
Is -5 prime? False
Is 4 prime? False
Is 29 prime? True
Is 100 prime? False
```

Explanation:

Converting Java code to Python helps improve flexibility and ease of development, as Python has simpler and more readable syntax. It allows programmers to understand the same logic in different programming languages and improves their problem-solving skills. This task also helps in learning how to adapt code structure, data types, and logic between languages, making it useful for real-world applications where multiple programming languages are used.

Question 3: Pseudocode to Python Implementation

Prompt:

Translate the given pseudocode for a sorting algorithm into executable Python code. Ensure that the Python implementation follows the same logic as the pseudocode, uses proper Python syntax, and handles lists of numbers correctly. Include sample input lists to test and validate that the program sorts the numbers as expected, and provide comments to explain each step of the code.

Algorithm: BubbleSort

Input: A list of numbers, arr

Output: Sorted list in ascending order

1. For i from 0 to length(arr) - 1
2. For j from 0 to length(arr) - i - 2
3. If arr[j] > arr[j + 1] then
4. Swap arr[j] and arr[j + 1]
5. End If
6. End For
7. End For
8. Return arr

Code:

```python
def bubble_sort(arr):
    """ Sorts a list of numbers using the Bubble Sort algorithm in ascending order."""
    n = len(arr)
    # Traverse through all array elements
    for i in range(n - 1):
        # Last i elements are already in place, so we don't need to check them
        for j in range(n - 1 - i):
            # Traverse the array from 0 to n-i-2
            # Swap if the element found is greater than the next element
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j] # Swap elements
    return arr
# Sample test cases
list1 = [64, 34, 25, 12, 22, 11, 90]
print(f"Original list: {list1}")
print(f"Sorted list: {bubble_sort(list1)}")
list2 = [5, 1, 4, 2, 8]
print(f"Original list: {list2}")
print(f"Sorted list: {bubble_sort(list2)}")
list3 = [1, 2, 3, 4, 5]
print(f"Original list: {list3}")
print(f"Sorted list: {bubble_sort(list3)}")
list4 = [5, 4, 3, 2, 1]
print(f"Original list: {list4}")
print(f"Sorted list: {bubble_sort(list4)}")
list5 = []
print(f"Original list: {list5}")
print(f"Sorted list: {bubble_sort(list5)}")
```

Output:

```
Original list: [64, 34, 25, 12, 22, 11, 90]
Sorted list: [11, 12, 22, 25, 34, 64, 90]
Original list: [5, 1, 4, 2, 8]
Sorted list: [1, 2, 4, 5, 8]
Original list: [1, 2, 3, 4, 5]
Sorted list: [1, 2, 3, 4, 5]
Original list: [5, 4, 3, 2, 1]
Sorted list: [1, 2, 3, 4, 5]
Original list: []
Sorted list: []
```

Explanation:

Translating pseudocode into Python helps bridge the gap between algorithmic thinking and actual programming. It allows you to take a language-independent logic and turn it into a working program, reinforcing understanding of loops, conditionals, and data manipulation. This task improves coding skills, ensures algorithms are correctly implemented, and makes it easier to test, debug, and adapt the logic for real-world applications.

Question 4: SQL to Pandas Query

Prompt:

Convert the given SQL query into an equivalent Pandas DataFrame operation in Python. Ensure that the logic of filtering and selecting columns remains the same while using proper Pandas syntax. The code should correctly retrieve the required subset of data from a DataFrame. Also, include a sample DataFrame and test the Pandas code to verify that it produces the same result as the SQL query.

SELECT name, salary

FROM employees

WHERE salary > 50000;

Code:

```python
import pandas as pd

# Create a sample DataFrame to represent the 'employees' table
employees_df = pd.DataFrame({
    'name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve'],
    'age': [30, 24, 35, 29, 40],
    'salary': [60000, 45000, 75000, 52000, 48000],
    'department': ['HR', 'IT', 'Finance', 'IT', 'HR']
})

print("Original DataFrame:")
display(employees_df)

# SQL Query: SELECT name, salary FROM employees WHERE salary > 50000;
# Equivalent Pandas operation:
# 1. Filter rows where 'salary' is greater than 50000
# 2. Select only the 'name' and 'salary' columns
high_earners_df = employees_df[employees_df['salary'] > 50000][['name', 'salary']]

print("\nDataFrame after SQL-equivalent operation (employees with salary > 50000):")
display(high_earners_df)
```

Output:

```
Original DataFrame:
```

	name	age	salary	department
0	Alice	30	60000	HR
1	Bob	24	45000	IT
2	Charlie	35	75000	Finance
3	David	29	52000	IT
4	Eve	40	48000	HR

```
DataFrame after SQL-equivalent operation (employees with salary > 50000):
```

	name	salary
0	Alice	60000
2	Charlie	75000
3	David	52000

Explanation:

Converting SQL queries into Pandas operations allows analysts and developers to work with data directly in Python without needing a database connection. It

helps leverage Python's powerful data manipulation and analysis capabilities while keeping the logic of filtering and selecting consistent with SQL. This task improves efficiency, makes data analysis more flexible, and ensures that results from SQL-based queries can be replicated in Python for reporting, visualization, or machine learning tasks.

Question 5: Real-Time Application: Algorithm Translation Across

Prompt:

Translate a given algorithm from one programming language into another while keeping its logic and functionality intact. Ensure that the translated version handles language-specific features such as syntax, data types, and library functions correctly. Test both the original and translated versions to verify that they produce the same results. Also, document any challenges faced during the translation, including differences in syntax, library support, or memory management. Include comments in the code and sample outputs to demonstrate correctness.

```python
def linear_search(arr, target):
    """
    Perform a linear search for the target in the given array.
    Parameters:
        arr (list): The list to search in.
        target: The value to search for.
    Returns:
        int: The index of the target if found, otherwise -1.
    """
    for i in range(len(arr)):
        if arr[i] == target:
            return i
    return -1

# Example usage
if __name__ == "__main__":
    my_list = [5, 3, 2, 8, 1]
    target_value = 2
    result = linear_search(my_list, target_value)

    if result != -1:
        print(f"Target {target_value} found at index: {result}")
    else:
        print(f"Target {target_value} not found in the list.")
```

Code:

```c
#include <stdio.h>
// Function to perform linear search
int linear_search(int arr[], int size, int target) {
    for (int i = 0; i < size; i++) {
        if (arr[i] == target) {
            return i; // Return the index if target is found
        }
    }
    return -1; // Return -1 if target is not found
}
int main() {
    int my_array[] = {5, 3, 2, 8, 1};
    int size = sizeof(my_array) / sizeof(my_array[0]);
    int target_value = 2;
    int result = linear_search(my_array, size, target_value);
    if (result != -1) {
        printf("Target %d found at index: %d\n", target_value, result);
    } else {
        printf("Target %d not found in the list.\n", target_value);
    }
    target_value = 10; // Test with a value not in the array
    result = linear_search(my_array, size, target_value);
    if (result != -1) {
        printf("Target %d found at index: %d\n", target_value, result);
    } else {
        printf("Target %d not found in the list.\n", target_value);
    }
    return 0;
}
```

```
# Compile the C code
!gcc linear_search.c -o linear_search

# Run the executable
!./linear_search
```

Output:

```
Target 2 found at index: 2
Target 10 not found in the list.
```

Explanation:

Translating an algorithm across programming languages helps developers understand how logic and functionality are implemented differently depending on language syntax, data types, and libraries. It improves problem-solving skills, teaches adaptation to language-specific features like memory management, and ensures that algorithms can be reused in multiple environments. This task is especially useful in real-world scenarios where companies maintain software in diverse languages, allowing consistent functionality across systems.